

“Orcs Arena”

Un Gioco Role-Play Text-Based in python

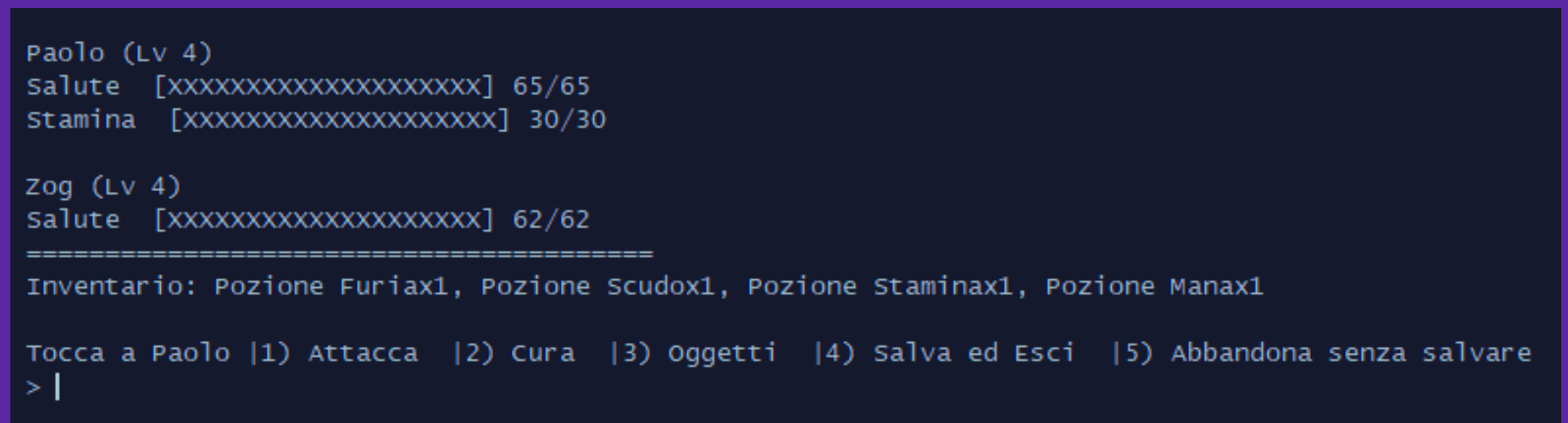


Progetto di Programmazione Object Oriented
di Thomas Andrea Porcari

Il Main

Contiene i loop del menù principale e il menù di combattimento; oltre che le funzioni di inizializzazione di una nuova partita, gestione dell'inventario e i metodi utili alla gestione degli eventi di gioco, quali il totalpartykill, la vittoria o l'uscita dal gioco con salvataggio

Realizzato senza l'utilizzo di librerie di alto livello e rispettando i tre principi della Programmazione ad Oggetti:



Flessibilità

I garantita dall'astrazione
che mi consente di
aggiungere classi
giocabili o oggetti senza
toccare il combat loop

Solidità

Ottenuta con
l'incapsulamento
evitando corruzioni
dirette delle variabili

Riutilizzo

Grazie al polimorfismo
che mi permette di
trattare party singoli o
doppi con la stessa
logica iterativa

Pozioni

Modulo adibito al Salvataggio e Caricamento della partita.
Al momento gestisce un solo salvataggio ma per il principio di

Furia



Si serializza lo stato sul file .txt scrivendo sequenzialmente righe in formato key=value con il numero di giocatori, le classi, i nomi, lo stage corrente e gli oggetti. Il file viene aperto in modalità di scrittura "w", sovrascrivendo l'eventuale contenuto precedente

&

Scudo Magico



Si apre il file in lettura "r", legge sequenzialmente le righe e ne estrae le coppie key=value facendo split sul carattere "=". Con questi parametri ricostruisce i giocatori, assegna i rispettivi oggetti, ripristina il loro livello corretto facendo levelup tante volte quanti sono gli stage superati e summona nuovamente un nemico

Separazione del main.py dalle classi complete:

Se non ci fosse questa factory il main dovrebbe importare ed essere accoppiato direttamente a ogni singola classe giocabile (Mage, Warrior, ecc.). Con la factory, il main riceve semplicemente un tipo astratto personaggi senza doversi preoccupare di come e quale classe concreta viene costruita.

Evitare codice duplicato:

Se dovessimo creare i personaggi nel main sia in modalità "Solo" sia in modalità "Coppia".
La factory centralizza la logica di creazione in un unico punto.

Rispettare SOLID e Scalabilità

Se decidiamo di aggiungere una nuova classe (es. Arciere), non devi toccare la logica del menu o del gioco nel main: aggiungi la classe in characters.py e una riga in creatore.py. Il resto del programma continua a funzionare inalterato.

Creatore

il pattern Factory CHE serve a
Incapsulare la creazione di un
personaggio permettendo di
dipendere su interfacce e non su
classi specifiche

Characters

Modulo che si occupa della gestione dei vari personaggi selezionabili dai giocatori con le loro features e proprietà.

Nello specifico il Mago e il Guerriero differiscono nella tipologia di attacco e nel nome della loro risorsa consumabile



Mage e Warrior



UNIVERSITÀ

LUM

GIUSEPPE
DEGENNARO

Nemici

Le entità base che combattono il giocatore. Possono solo ricevere danno e attaccare secondo il Pattern Behavioral in “strategianemica.py”. Delle implementazioni future potrebbero consistere nello sviluppo di diverse tipologie di nemici con attacchi differenti o di Boss ogni tot stages



Orco



UNIVERSITÀ
LUM

GIUSEPPE
DEGENNARO

Strategia Nemica

attaccarandom viene scelto un
giocatore a caso

conmenovita viene scelto il giocatore
con meno hp

conpiuvita viene scelto il giocatore
con più hp

Behavioral Design Pattern Strategy

Una semplice meccanica di IA dei nemici per stabilire chi targhettare durante il combattimento. Con l'introduzione di nemici più complessi con tipologie di attacchi differenti sarà possibile aumentare le variabili e le scelte adattando lo stile di gioco del nemico alle più disparate situazioni

Saving and Loading

Modulo adibito al Salvataggio e Caricamento della partita.
Al momento gestisce un solo salvataggio ma per il principio di

Salvataggio



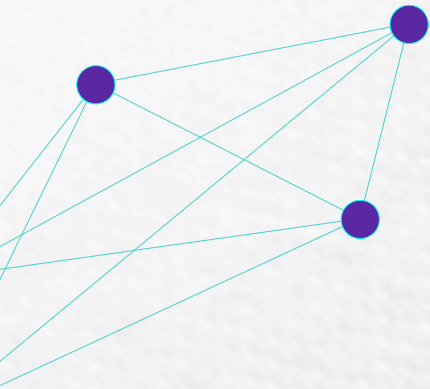
Si serializza lo stato sul file .txt scrivendo sequenzialmente righe in formato key=value con il numero di giocatori, le classi, i nomi, lo stage corrente e gli oggetti. Il file viene aperto in modalità di scrittura "w", sovrascrivendo l'eventuale contenuto precedente

&

Caricamento



Si apre il file in lettura "r", legge sequenzialmente le righe e ne estrae le coppie key=value facendo split sul carattere "=". Con questi parametri ricostruisce i giocatori, assegna i rispettivi oggetti, ripristina il loro livello corretto facendo levelup tante volte quanti sono gli stage superati e summona nuovamente un nemico





Letture e Scrittura di un file binario

Per il requisito del file binario, ho scelto di introdurre una funzione di hall of fame che memorizzi i 10 migliori punteggi dei giocatori. La hall of fame persiste nel file binario che viene aperto in modalità 'wb' e 'rb'. Questo garantisce persistenza reale non volatile e accesso a dimensione fissa.

```
x=x=x=x= Hall of fame =x=x=x=x
1) Arthur Pendragon - Stage 10 - 1085 punti
2) Conan e Gandalf - Stage 10 - 1040 punti
3) Legolas - Stage 8 - 830 punti
4) Gimli e Elrond - Stage 7 - 750 punti
5) Geralt - Stage 6 - 625 punti
6) Boromir - Stage 4 - 410 punti
7) Frodo e Sam - Stage 2 - 215 punti
x=x=x=x=x=x=x=x=x=x=x=x=x=x=x=x
```